Intelligent Exam Proctoring Logic & Blueprint Document

Course Code: 25SC1306E | Computational Foundations for Artificial Intelligence

This reference specification presents the comprehensive implementation of an **Intelligent Exam Proctoring Framework**. The layout leverages core Artificial Intelligence paradigms defined across course objectives, substituting the handbook's robot routing parameters with live testing evaluation controls.

Syllabus Outcome Assignment Index

Outcome	Syllabus Competency Mapping Target	Program Execution Mapping Vector
CO1	Formulate states, actions, parameters, transitions, and step reasoning histories.	State space initialization using typed objects and traceable operational arrays.
CO2	Implement uninformed/informed structures with admissible heuristic evaluation boundaries.	A* search algorithms evaluating real-time anomaly trajectories across risk nodes.
CO3	Model Constraint Satisfaction Problems (CSP) through backtracking parameters.	Hardware layout clearance and functional data pipeline integrity verification.
CO4	Design decisions in sequential/adversarial variations with bounded visibility levels.	Minimax verification algorithms assessing malicious path metrics against triggers.
CO5	Apply reasoning under environmental uncertainty via dynamic probability distributions.	Bayes' Rule processing modules extracting risk facts from ambiguous hardware metrics.
CO6	Synthesize sub-module elements into explainable operational reasoning logs.	Pipeline coordination systems presenting auditable tracking arrays for override validation.

Python Code Blueprint

```python
import dataclasses
from typing import List, Dict, Set, Tuple
import heapq
import math
import random

# ======================================================================
# CO1: PROBLEM FORMULATION & KNOWLEDGE REPRESENTATION
# Formulating the environment snapshot fields using typed dataclasses.
# ======================================================================

@dataclasses.dataclass(frozen=True)
class ProctoringState:
    head_posture: int   # Displace index from target workspace view
    audio_level: float  # Captured input ambient sound (decibel scale)
    tab_switches: int   # Count parameters for context transitions

class ExamProctorAgent:
    def __init__(self, PEAS_description: dict):
```

```
        self.peas = PEAS_description
        self.reasoning_log: List[str] = []

    def log_reasoning(self, step_trace: str):
        self.reasoning_log.append(step_trace)

# ======================================================================
# CO2: INFORMED SEARCH METHODOLOGY & HEURISTIC VERIFICATION
# Graph models calculating metric expansion paths under open/closed sets.
# ======================================================================

class ProctoringSearchSpace:
    def __init__(self):
        self.graph: Dict[str, List[Tuple[str, float]]] = {
            "Normal": [("Suspicious_Look", 2.0), ("Noise_Spike", 1.5)],
            "Suspicious_Look": [("Escalated_Risk", 3.0), ("Normal", 1.0)],
            "Noise_Spike": [("Escalated_Risk", 2.5), ("Normal", 1.0)],
            "Escalated_Risk": [("Flagged_Violation", 4.0)]
        }

    def get_neighbors(self, state: str) -> List[Tuple[str, float]]:
        return self.graph.get(state, [])

    @staticmethod
    def heuristic(current_state: str, goal_state: str) -> float:
        h_values = {
            "Normal": 4.0,
            "Suspicious_Look": 2.0,
            "Noise_Spike": 2.0,
            "Escalated_Risk": 1.0,
            "Flagged_Violation": 0.0
        }
        return h_values.get(current_state, 99.0)

    def a_star_search(self, start: str, goal: str) -> Tuple[List[str], float]:
        open_set = []
        heapq.heappush(open_set, (0 + self.heuristic(start, goal), 0, start, [start]))
        closed_set: Set[str] = set()

        while open_set:
            f, g, current, path = heapq.heappop(open_set)
            if current in closed_set: continue
            if current == goal: return path, g
            closed_set.add(current)

            for neighbor, cost in self.get_neighbors(current):
                if neighbor not in closed_set:
                    new_g = g + cost
                    new_f = new_g + self.heuristic(neighbor, goal)
                    heapq.heappush(open_set, (new_f, new_g, neighbor, path + [neighbor]))
        return [], float('inf')

# ======================================================================
# CO3: CONSTRAINT SATISFACTION PROCESSES (CSP)
# Checks domain configuration alignment against structural runtime limits.
# ======================================================================

class ProctoringCSP:
    def __init__(self):
        self.variables = ["Primary_Webcam", "Microphone", "Secondary_Cam"]
        self.domains = {
            "Primary_Webcam": ["Clear", "Obstructed", "Low_Light"],
            "Microphone": ["Active_Feed", "Muted"],
            "Secondary_Cam": ["Wide_Angle_Clear", "No_Feed"]
        }
```

```
    def is_consistent(self, assignment: Dict[str, str]) -> bool:
        if "Primary_Webcam" in assignment and assignment["Primary_Webcam"] in ["Obstructed",
"Low_Light"]:
            return False
        if "Microphone" in assignment and assignment["Microphone"] == "Muted":
            return False
        if "Secondary_Cam" in assignment and assignment["Secondary_Cam"] == "No_Feed":
            return False
        return True

    def backtracking_search(self, assignment: Dict[str, str] = None) -> Dict[str, str]:
        if assignment is None: assignment = {}
        if len(assignment) == len(self.variables): return assignment

        unassigned = [v for v in self.variables if v not in assignment]
        var = unassigned[0]

        for value in self.domains[var]:
            local_assignment = assignment.copy()
            local_assignment[var] = value
            if self.is_consistent(local_assignment):
                result = self.backtracking_search(local_assignment)
                if result is not None: return result
        return None

# ======================================================================
# CO4: GAME THEORY & COMPREHENSIVE DECISION STRATEGIES
# Simulates metric variations via game optimization bounds.
# ======================================================================

class ProctoringDecisionGame:
    @staticmethod
    def minimax_decision(depth: int, is_agent_maximizing: bool) -> int:
        if depth == 0: return random.choice([10, 45, 80])

        if is_agent_maximizing:
            max_eval = -math.inf
            for action_move in [1, 2]:
                evaluation = ProctoringDecisionGame.minimax_decision(depth - 1, False)
                max_eval = max(max_eval, evaluation)
            return max_eval
        else:
            min_eval = math.inf
            for student_move in [1, 2]:
                evaluation = ProctoringDecisionGame.minimax_decision(depth - 1, True)
                min_eval = min(min_eval, evaluation)
            return min_eval

# ======================================================================
# CO5: PROBABILISTIC LOGIC & UNCERTAINTY MANAGEMENT
# Normalizes fluctuating inputs via formal conditional algorithms.
# ======================================================================

class UncertaintyProbabilisticReasoning:
    @staticmethod
    def calculate_cheating_probability(has_anomaly_triggered: bool) -> float:
        p_cheating = 0.05
        p_clean = 0.95
        p_anomaly_given_cheating = 0.90
        p_anomaly_given_clean = 0.15

        if has_anomaly_triggered:
            p_anomaly = (p_anomaly_given_cheating * p_cheating) + (p_anomaly_given_clean * p_clean)
            return round((p_anomaly_given_cheating * p_cheating) / p_anomaly, 4)
```

```
        return 0.002

# ======================================================================
# CO6: DISCRETE PIPELINE PIPING AND VERIFIED AUDIT TRAILS
# Orchestrates modules to construct traceable validation output fields.
# ======================================================================

def run_intelligent_exam_proctor_pipeline():
    peas_config = {
        "Performance": "Maintain exam integrity while minimizing false-positive flags",
        "Environment": "Web browser interface, camera profile, audio feeds",
        "Actuators": "Automated warnings, human-proctor dashboard updates",
        "Sensors": "Computer vision posture matrices, audio volume capture array"
    }

    agent = ExamProctorAgent(peas_config)
    agent.log_reasoning("[CO1] Formulated target environment bounds via PEAS infrastructure.")

    current_telemetry = ProctoringState(head_posture=35, audio_level=72.5, tab_switches=1)
    agent.log_reasoning(f"[CO1] Parsed tracking parameters. Input posture displacement variance
observed.")

    csp_verifier = ProctoringCSP()
    validation_assignment = csp_verifier.backtracking_search()
    if validation_assignment:
        agent.log_reasoning(f"[CO3] Structural resource matching parameters resolved:
{validation_assignment}")

    search_space = ProctoringSearchSpace()
    escalation_path, total_risk_cost = search_space.a_star_search("Normal", "Flagged_Violation")
    agent.log_reasoning(f"[CO2] Calculated risk routing progression through A*: {escalation_path}.")

    raw_anomaly_detected = current_telemetry.head_posture > 30 or current_telemetry.audio_level > 65.0
    posterior_risk =
UncertaintyProbabilisticReasoning.calculate_cheating_probability(raw_anomaly_detected)
    agent.log_reasoning(f"[CO5] Probabilistic factor established: P(Violation | Flag) =
{posterior_risk * 100}%.")

    optimal_game_policy = ProctoringDecisionGame.minimax_decision(depth=2, is_agent_maximizing=True)
    agent.log_reasoning(f"[CO4] Automated variance decision threshold finalized:
{optimal_game_policy}.")

    print("System compiled validation pipelines successfully.")
```